Case study: security is a second-class citizen in pi

Stacia

An attempt to understand pi's permission model surfaced a structural flaw in how pi approaches security.

Status: case study plus the decisions it forced.

How it started: building to understand

This began as a way to understand how pi handles permissions: a tool-call **sieve gate**, built to learn the domain rather than to ship a product.

It intercepts each tool call, runs it through an ordered *rejection* sieve (auto-deny with a teaching reason), then an ordered *approval* sieve (auto-allow), and sends anything left to a human checkpoint with a plain-language summary. It hooks pi's `tool_call` event and works: it denies, approves, and prompts on real commands.

The pivot: subagents don't share the parent's extension context

Checking whether the gate would cover **subagent** activity revealed that it usually does not. A subagent runs in its own session, and that session only enforces the extensions **it** loaded. The parent's gate does not automatically extend to a child.

That changed the question from "what rules should the gate apply?" to:

> *Can a security gate in pi even be* guaranteed to run *on the work being done?*

Answering it required reading source.

The investigation

Read at depth: three subagent extensions (`@quintinshaw/pi-dynamic-workflows`, `nicobailon/pi-subagents`, `tintinweb/pi-subagents`), `@gotgenes/pi-permission-system`'s subagent integration, and the pi harness itself (the tool-call dispatch path and the resource loader that decides which extensions a session loads). Three findings.

1. pi's tool boundary is a cooperative, mutable chain

pi exposes one extension-facing seam at the tool boundary (`agent-session.js:214`): a `tool_call` hook before execution, a `tool_result` hook after. Dispatch (`runner.emitToolCall`) walks **all bound extensions in load order**. A handler may return `{ block, reason }` or **mutate the tool input in place with no re-validation**, and the **first** block wins. Two properties follow:

- **Enforcement is per-session and opt-in.** The handler set is whatever *that* session bound. A session that did not load the gate has no gate.
- **The gate is a peer, not an authority.** It shares a mutable input with other extensions, order-dependent, so a sibling loaded *after* it can rewrite a command the gate already approved.

2. Whether a subagent loads the gate varies wildly by extension

The three answer "does the parent's policy run in the child?" differently:

- **`dynamic-workflows`: no.** Children are created with `noExtensions: true` hard-coded; the gate never loads. Tool granting is whole-tool allow/deny at spawn (`applyToolPolicy`), no command-level nuance. It leans on git-worktree isolation, which is damage confinement, not access control.
- **`nicobailon`: only inside one ecosystem.** It is *subprocess*-based, the only one with a real process boundary, but exposes no hooks a general policy can bind to. It targets `pi-permission-system` and misses env wiring (`PI_SUBAGENT_PARENT_SESSION`) others would need. Outside that ecosystem there is no viable integration path.
- **`tintinweb`: yes, by default, and scopably.** It loads host extensions into children (`extensions: true`) and calls `bindExtensions`, so the gate runs per-call; loading is declaratively scopable (`extensions: [stacia-sieve-gate]`). It stays in-process (no boundary) and reloads everything per child by default, but it is the only one that gives a policy real purchase inside subagents.

Same policy, unchanged: fully enforced, partially enforced, or entirely absent, depending on which subagent extension is in use and how it was configured.

3. `noExtensions` is a scalpel, which worsens the inconsistency

`noExtensions: true` does not mean "no extensions in the child" (`resource-loader.js:267`). It drops only the auto-discovered set; `additionalExtensionPaths` and inline `extensionFactories` still load. So `noExtensions: true` plus `extensionFactories: [gate]` loads *only* the gate. The capability to guarantee a child is gated exists in the harness. It is not wired to any consistent, first-class control, so each subagent extension reinvents or ignores it.

The conclusion: a structural flaw

The finding is about pi, not any one extension.

Security in pi is a second-class element, an extension composed with peer extensions, so it can never be guaranteed structurally. The precise failure is not "pi can't stop every attack" (no in-process model can) but:

> *pi cannot guarantee the security layer is even* ***present and authoritative*** *on a given session.*

The gate can be unloaded (`isolated` / `noExtensions`; the same `noExtensions` is also the scalpel of finding 3), it competes with peers (load order), and its decisions can be mutated after approval. A security posture built on pi is defeatable by *configuration* and by *siblings*.

Which risks are pi's and which are common to the paradigm:

- **pi-specific (the differentiators).** A sibling handler mutating an approved input, and a child spawned that never loads the gate. Both come from the cooperative, opt-in-per-session model. A harness with a single always-present authoritative decision point removes both.
- **Paradigm-inherent (shared with any in-process agent, including Claude Code).** A Turing-complete shell as an escape hatch, two-step laundering with no information-flow control, human-in-the-loop fatigue and broad-grant abuse, and rules stored in agent-editable files. These are present in any harness and close only with a real boundary (OS sandbox, separate user, credential isolation, information-flow control).

The differentiator is entirely pi-specific and architectural. pi does not need an opinion on *how* to implement security. It lacks a **first-class, unified substrate**: a harness-owned, always-present, authoritative permission decision point that security *and* ordinary extensions bind to as policy, instead of racing as equal peers over a mutable input. Without it, security on pi is structurally unguaranteeable, and the burden of closing that falls on userspace.

This is a potential death-knell for adopting pi. The paradigm-inherent vectors are not pi's fault. The structural one, no guarantee the gate is present and final, is, and it cannot be patched around from an extension.

Decisions and mitigations

The pragmatic path on pi:

- **Adopt `tintinweb/pi-subagents`**, the only substrate where the gate runs per-call in subagents, scoped to `extensions: [stacia-sieve-gate]` to keep blast radius small.
- **Forward subagent fall-throughs to the parent UI** via a `Symbol.for()` broker (child sessions have no UI), with a narrowly-scoped grant cache to blunt fatigue.
- **Gate the spawn**, denying `Agent` calls that would run ungated (`isolated: true`, opt-out `subagent_type`s), since every bypass the parent can reach is itself a tool call it sees. This blocks *unintentional* ungated spawns — a naive request or a careless config. It does **not** stop an adversarial sibling: per finding 1, a handler loaded after the gate can set `isolated: true` back *after* approval, and the ungated child spawns anyway. The spawn gate would hold only if the gate were guaranteed to run last in the chain (validating the final input) — exactly the load-order authority pi cannot grant.
- **Protect the preconditions**, treating writes to `.pi/agents/**`, `~/.pi/agent/extensions/**`, pi settings/auth, and pi-re-invoking `bash` as top-sensitivity.

Every item is userspace mitigation that raises cost and coverage; none is a structural guarantee. Uncloseable under this design without a harness change or a real boundary: post-approval mutation, the shell-breadth escape hatch, and the absence of information-flow control.

Evidence index

- pi tool hooks / dispatch: `agent-session.js:214` (`beforeToolCall` to `emitToolCall`); `runner.emitToolCall` (load order, first-block short-circuit, mutable input, no re-validation).

- `noExtensions` semantics: `resource-loader.js:267`; `additionalExtensionPaths` and `extensionFactories` survive it.
- dynamic-workflows: `agent.ts` (`noExtensions: true`; `applyToolPolicy`; `DEFAULT_EXCLUDED_SUBAGENT_TOOL` `session.resourceLoader` reachable only to programmatic embedders.
- tintinweb: default `extensions: true` (`agent-types.ts`, `default-agents.ts`); loader-level `extensionsOverride`; `isolated` is an `Agent` tool param (`index.ts`) forcing `extensions === false` (`agent-runner.ts:524`); `bindExtensions` fires `session_start`; child has no UI (`ctx.hasUI === false`).
- nicobailon: subprocess model; missing `PI_SUBAGENT_PARENT_SESSION`; `tools:` CSV allowlist.
- pi-permission-system: native `@gotgenes/pi-subagents` integration (process-global registry plus ask-forwarding); two-layer visibility/policy model.